

PROYECTO NANOFILES

Curso 25/26; Convocatoria Enero-Junio
Redes de Comunicaciones
Subgrupo 9.1
Profesor: Juan José Pujante Bernal

Pedro Blaya Pascual (48853636A)
Daniel Pozas Ruiz (26548389H)

ÍNDICE

ÍNDICE.....	1
1. INTRODUCCIÓN.....	2
2. PROTOCOLOS DISEÑADOS.....	2
2.1. Directorio.....	2
2.1.1 Mensajes y ejemplos.....	2
2.1.1.1 Tipos y descripción de los mensajes.....	2
2.1.2 Autómatas.....	6
2.2. Peers.....	8
2.2.1 Mensajes y ejemplos.....	8
2.2.2 Autómatas.....	10
2.2.2.2 Automata peer-peer-servidor.....	11
3. Mejoras.....	11

1. INTRODUCCIÓN

En este documento se expone cómo hemos diseñado los distintos mensajes de comunicación, tanto peer-to-peer como un peer con el directorio. También se incluye una representación gráfica de los autómatas con estados finitos.

En cuanto a las mejoras, hemos considerado la implementación de todas las posibles.

2. PROTOCOLOS DISEÑADOS

2.1. Directorio.

2.1.1 Mensajes y ejemplos.

Para definir el protocolo de comunicación con el Directorio, vamos a utilizar mensajes textuales con formato “campo:valor”. El valor que tome el campo “operation” (código de operación) indicará el tipo de mensaje y por tanto su formato (que campos vienen a continuación).

2.1.1.1 Tipos y descripción de los mensajes

Mensaje: **ping**

Sentido de la comunicación: Cliente -> Directorio

Descripción: contactar con el servidor de directorio para comprobar que está a la escucha y que utiliza un protocolo compatible con el del peer.

Ejemplo:

```
operation:ping\n
protocolid:12345678A\n
```

Mensaje: **ping_ok**

Sentido de la comunicación: Cliente <- Directorio

Descripción: indicar que el protocolo que acompaña al mensaje ping del cliente es igual al que espera el servidor(Directorio).

Ejemplo:

```
operation:ping_ok\n
```

Mensaje: **ping_error**

Sentido de la comunicación: Cliente <- Directorio

Descripción: indicar que el protocolo que acompaña al mensaje ping del cliente no es igual al que espera el servidor(Directorio)

Ejemplo:

```
operation: ping_error\n
```

Mensaje: **dirfiles_req**

Sentido de la comunicación: Cliente -> Directorio

Descripción: indica al directorio que le comience a mandar el listado de ficheros del que dispone.

Ejemplo:

operation:dirfiles_req\n

Mensaje: **dirfiles_rep**

Sentido de la comunicación: Cliente <- Directorio

Descripción: listado de ficheros donde para cada fichero se indica su nombre, tamaño y hash. Se tiene en cuenta que el listado puede acomodarse en más de un datagrama.

Ejemplo:

operation:dirfiles_rep\n
block_number:i_block\n
file:name,size,hash\n
file:name,size,hash\n
.
.
.
file:name,size,hash\n

Mensaje: **dirfiles_ack**

Sentido de la comunicación: Cliente -> Directorio

Descripción: indica al directorio que el bloque de datos i ha sido recibido correctamente

Ejemplo:

operation:dirfiles_ack\n
ack_number:i_ack\n

Mensaje: **dirfiles_ok**

Sentido de la comunicación: Cliente <- Directorio

Descripción: indica al cliente que ya ha enviado todos los bloques en los que se ha dividido el listado de ficheros.

Ejemplo:

operation:dirfiles_ok\n

Mensaje: **dirfiles_error**

Sentido de la comunicación: Cliente <- Directorio

Descripción: indica al cliente que se ha interrumpido el proceso de envío de paquetes con la información del listado de ficheros. Este error viene indicado por el campo error_info.

Ejemplo:

operation:dirfiles_error\n
error_info:string(una linea)\n

Mensaje: **serve**

Sentido de la comunicación: Cliente -> Directorio

Descripción: pide al directorio registrarse como un servidor de ficheros que escuche conexiones en un puerto TCP, y registrar dicha dirección de escucha (IP y puerto) en el directorio, asociándola al nickname del peer.

Ejemplo:

```
operation:server\n
serve:name,IP,puerto\n
```

Mensaje: **serve_ok**

Sentido de la comunicación: Cliente <- Directorio

Descripción: indica que se ha podido registrar como servidor ese peer

Ejemplo:

```
operation:serve_ok\n
```

Mensaje: **peers**

Sentido de la comunicación: Cliente -> Directorio

Descripción: pide el censo de peers registrados como servidores en el directorio.

Ejemplo:

```
operation:peers\n
```

Mensaje: **peers_ok**

Sentido de la comunicación: Cliente <- Directorio

Descripción: muestra el censo de peers registrados como servidores en el directorio.

El listado indica para cada par su nickname, IP y puerto TCP de escucha.

Ejemplo:

```
operation:serve_ok\n
serve:name,IP,puerto\n
serve:name,IP,puerto\n
.
.
.
serve:name,IP,puerto\n
```

Mensaje: **dirdl_req**

Sentido de la comunicación: Cliente -> Directorio

Descripción: solicita al directorio la descarga de un archivo cuyo hash contenga el substring pasado como parámetro. Falla si hay varios archivos que lo contengan.

Ejemplo:

```
operation:dirdl_req\n
subhash:subhash\n
```

Mensaje: **dirdl_reply**

Sentido de la comunicación: Cliente <- Directorio

Descripción: responde con los datos, codificados en Base64, de uno de los chunks del archivo (en caso de que este se divida en varios) identificado unívocamente por el subhash.

Ejemplo:

```
operation:dirdl_reply\n
block_number:block_number\n
```

data:data(Base64)\n

Mensaje: dirdl_ok

Sentido de la comunicación: Cliente <- Directorio

Descripción: notifica al cliente de que se han enviado exitosamente todos los bloques del archivo cuya descarga había sido solicitada, incluyendo el nombre del archivo, tamaño y su hash completo en el envío.

Ejemplo:

```
operation:dirdl_ok\nfile_size:filesize\nfile_name:filename\nsubhash:fullhash\n
```

Mensaje: dirdl_error

Sentido de la comunicación: Cliente <- Directorio

Descripción: notifica al cliente de que la descarga no se ha podido llevar a cabo debido a errores inesperados o errores contemplados como, por ejemplo, la duplicidad del substring pasado como parámetro en varios hashes. En este último caso envía un string de un línea con información adicional.

Ejemplo:

```
operation:dirdl_error\nerror_info:string(una linea)\n
```

Mensaje: quit

Sentido de la comunicación: Cliente -> Directorio

Descripción: cierra la aplicación. Si el servidor estaba activo, se detiene y se solicita la baja al directorio.

Ejemplo:

```
operation:quit\nnickname:name\n
```

Mensaje: quit_ok

Sentido de la comunicación: Cliente <- Directorio

Descripción: notifica al cliente de que su solicitud de cierre seguro se ha completado exitosamente.

Ejemplo:

```
operation:quit_ok\n
```

Mensaje: quit_error

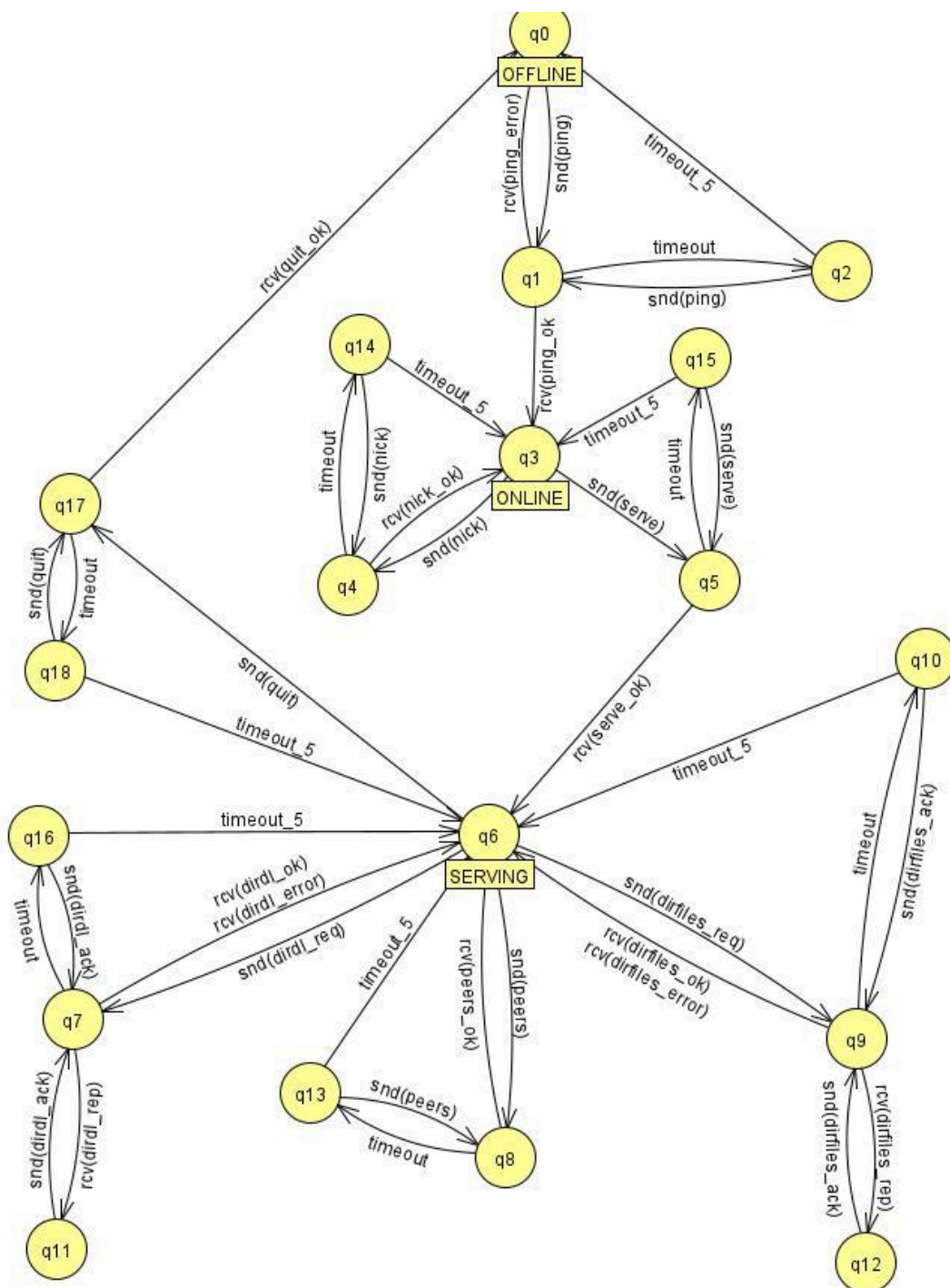
Sentido de la comunicación: Cliente <- Directorio

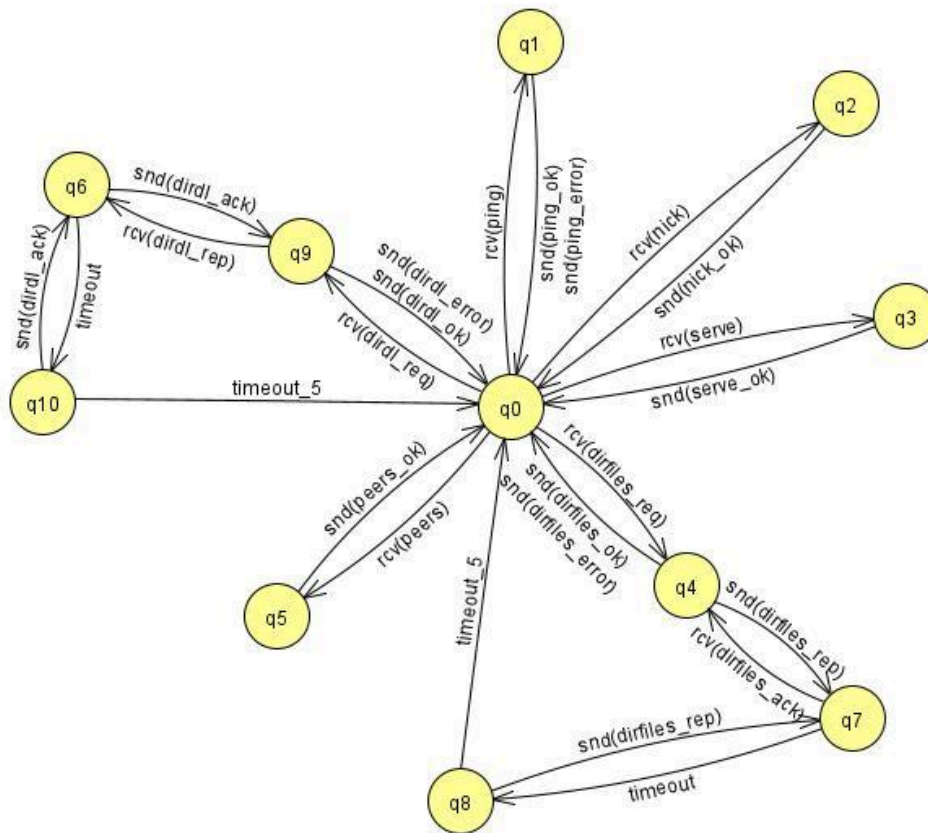
Descripción: notifica al cliente de que su solicitud de cierre seguro no se ha podido completar exitosamente

2.1.2 Autómatas.

Estos son los esquemas de la lógica de los autómatas implementados en el proyecto.

2.1.2.1 . Autómata peer-directorio



2.1.2.2 Autómata directorio-peer

2.2. Peers.

2.2.1 Mensajes y ejemplos.

Mensaje: **peerfiles_req** (opcode = 1)

Sentido de la comunicación: Servidor <- Cliente

Descripción: pide listar los ficheros disponibles en un peer servidor de ficheros dado por su nickname.

Ejemplo:

Opcode
(1 byte)
0x01

Mensaje: **peerfiles_reply** (opcode = 2)

Sentido de la comunicación: Servidor -> Cliente

Descripción: indica que se han obtenido los ficheros del servidor de ficheros y manda la lista serializada.

Ejemplo:

Opcode	Longitud lista	Lista serializada
(1 byte)	(4 bytes)	(n bytes)
0x02	0x(n)	ej

Mensaje: **peerfiles_error** (opcode = 8)

Sentido de la comunicación: Servidor -> Cliente

Descripción: indica que se el peer no se encuentra como servidor de ficheros.

Ejemplo:

Opcode
(1 byte)
0x08

Mensaje: **peerdl_req** (opcode = 3)

Sentido de la comunicación: Servidor <- Cliente

Descripción: descargar un fichero de entre los ficheros disponibles en un peer dado

por su nickname. El fichero será identificado por una subcadena del hash. Estas descargas se harán a través de chunks.

Ejemplo:

Opcode	Longitud hash	Hash
(1 byte)	(4 bytes)	(n bytes)
0x03	0x(n)	ej

Mensaje: **peerdl_reply** (opcode = 4)

Sentido de la comunicación: Servidor -> Cliente

Descripción: devuelve la información del archivo en caso de que este haya sido identificado unívocamente por su subhash.

Ejemplo:

Opcode	File Size	Tamaño nombre	Nombre	Tamaño hash	Hash
(1 byte)	(8 bytes)	(4 bytes)	(m bytes)	(4 bytes)	(l bytes)
0x04	0x(n)	0x(m)	ej	0x(l)	ej

Mensaje: **peerdl_file** (opcode = 5)

Sentido de la comunicación: Servidor <- Cliente

Descripción: el cliente solicita la descarga del x bytes del fichero a partir del offset y. Clave pues la gestión de chunks en este caso la hará el cliente y no el servidor, al contrario que en dirdl.

Ejemplo:

Opcode	Tamaño hash	Hash	Offset	Length
(1 byte)	(4 bytes)	(n bytes)	(8 bytes)	(8 bytes)
0x05	0x(n)	ej	0x(m)	0x(l)

Mensaje: **peerdl_data** (opcode = 6)

Sentido de la comunicación: Servidor -> Cliente

Descripción: responde con los datos solicitados previamente por la operación peerdl_file.

Ejemplo:

Opcode	Longitud data	Data
(1 byte)	(4 bytes)	(n bytes)
0x06	0x(n)	ej

Mensaje: **peerdl_error** (opcode = 7)

Sentido de la comunicación: Servidor -> Cliente

Descripción: notifica al cliente de que no se ha podido llevar a cargo la distancia del fichero solicitado, en casos concretos, como duplicidad del subhash manda un mensaje informativo adjunto.

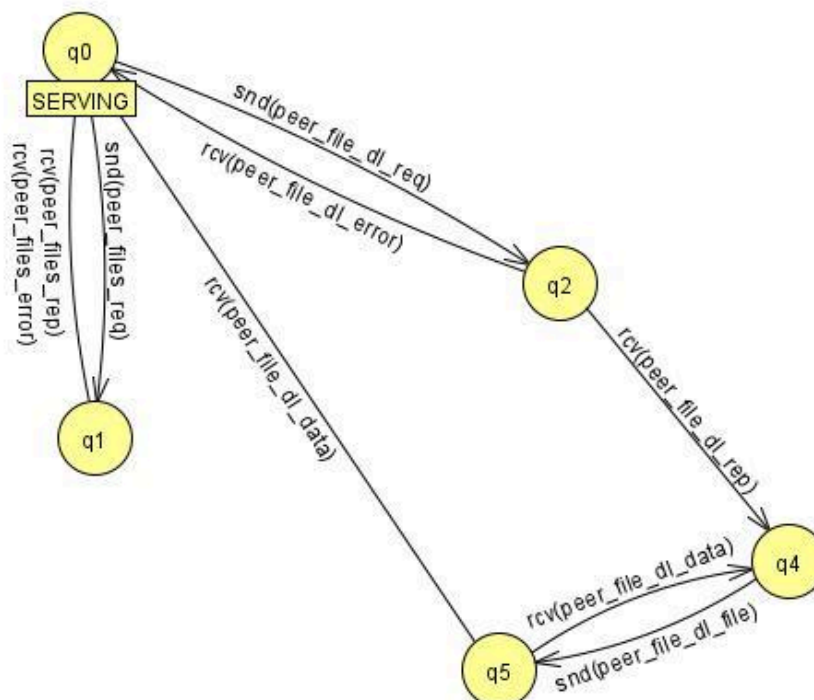
Ejemplo:

Opcode	Tamaño info	Info
(1 byte)	(4 bytes)	(n bytes)
0x07	0x(n)	ej

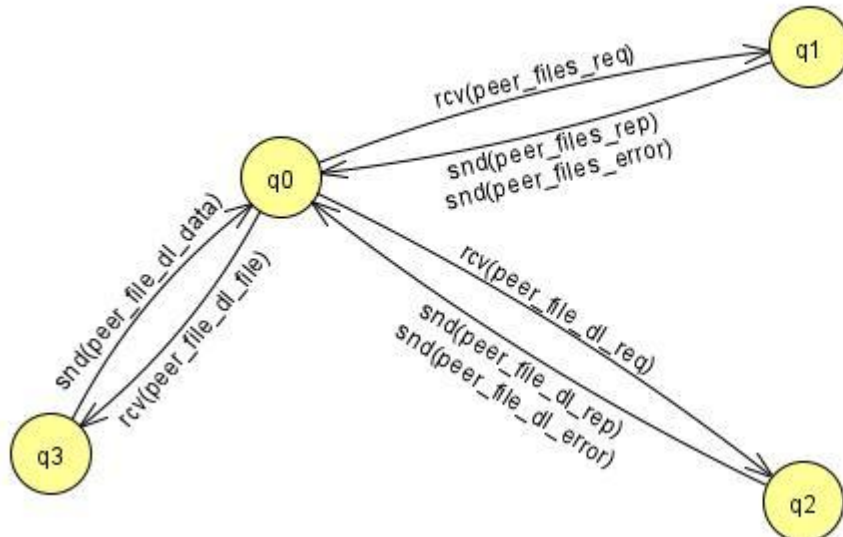
2.2.2 Autómatas.

A pesar de haber implementado el comando peerdl * , aquí no incluimos sus estados porque son los mismos que los de peerdl con la salvedad de que los rcv(peer_file_dl_data) provienen de peers distintos.

2.2.2.1 Automata peer-peer-cliente



2.2.2.2 Automata peer-peer-servidor



3. Mejoras

Hemos implementado todas las mejoras.

dirdl:

Para el dirdl hablaremos directamente del dirdl ampliado ya que lo hicimos todo a la vez. Hemos seguido un Stop And Wait para implementarlo, el servidor divide en chunks, los va enviando, hasta que no reciba el correspondiente ACK no envía el siguiente. Cuando termina de enviar el último lanza un mensaje de control para que el cliente lo sepa.

Una consideración importante es que el directorio mientras espera los acks tiene un timeout porque podría pasar que el cliente se diera por vencido y el directorio siguiera a la espera de un ack que nunca llegará. Si eso llegase a pasar el directorio quedaría encerrado en esa lógica y cuando fuésemos a ejecutar otro comando obtendremos datos erróneos. Para evitar guardar información duplicada se utilizan variables para guardar el ack que esperamos. Si recibimos del directorio una respuesta que ya hemos almacenado dicho paquete se descarta y se vuelve a enviar el ack del último paquete recibido.

dirfiles:

La ampliación de dirfiles ha sido bastante sencilla. A pesar de que el contenido de los mensajes de este comando son diferentes, la lógica de trocear la información que queremos

mandar y su sistema es el mismo que el del dirdl ampliado. En cuanto al contenido del envío, hemos limitado a 9 el número de ficheros que podemos mandar para así no sobrepasar el tamaño de los paquetes.

peerdl *:

La ampliación del peerdl fue bastante sencilla, solo tuvimos que gestionar que la descarga de chunks fuera recorriendo la lista de servidores, usamos una variable peerIndex que los va recorriendo. Cambiamos la forma de escribir en el archivo FileOutputStream por RandomAccessFile, para que nos fuera más sencillo seguir el orden que marcaba la variable offset. En caso de que alguno de los servidores fallara al mandar un chunk se le eliminaría de la lista de servidores y se volvería a intentar descargar el chunk desde otro servidor diferente.

quit:

Para el comando quit simplemente implementamos la función unregisterFileServer de directoryConnector, que indica al directorio que ha de eliminar al peer de su lista. Mediante la gestión de excepciones en NFServer conseguimos implementar el método stopFileServer de manera limpia y adecuada, el controlador del peer llama este método para llevar a cabo la segunda funcionalidad de la operación, parar el hilo servidor.

prompt:

Se ha cambiado el prompt básico de la terminal de la aplicación. En cuanto a la ejecución de esta mejora no hay mucho que comentar ya que solo ha consistido en cambiar una impresión de pantalla.

4. Capturas de Wireshark

En nuestro ejemplo se ejecutan los comandos ping, serve, peerfiles y quit; en ese orden. Solo analizaremos los mensajes UDP pues se pide un ejemplo de comunicación exclusivamente con el directorio en el enunciado. La razón de ejecutar el comando peerfiles es que internamente usa comunicación con el directorio (operación peers) y nos ha parecido interesante ponerlo.

captura_wiresh

Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda

Aplique un filtro de visualización ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	UDP	80	51983 → 6868 Len=38
2	0.000806451	127.0.0.1	127.0.0.1	UDP	61	6868 → 51983 Len=19
3	3.783978516	127.0.0.1	127.0.0.1	UDP	77	51983 → 6868 Len=35
4	3.784784906	127.0.0.1	127.0.0.1	UDP	83	6868 → 51983 Len=41
5	8.090794368	127.0.0.1	127.0.0.1	UDP	59	51983 → 6868 Len=17
6	8.113473764	127.0.0.1	127.0.0.1	UDP	115	6868 → 51983 Len=73
7	8.118806031	127.0.0.1	127.0.0.1	TCP	74	44210 → 10000 [SYN] Seq=
8	8.118839338	127.0.0.1	127.0.0.1	TCP	74	10000 → 44210 [SYN, ACK]
9	8.118861183	127.0.0.1	127.0.0.1	TCP	66	44210 → 10000 [ACK] Seq=
10	8.121009946	127.0.0.1	127.0.0.1	TCP	67	44210 → 10000 [PSH, ACK]
11	8.121025650	127.0.0.1	127.0.0.1	TCP	66	10000 → 44210 [ACK] Seq=
12	8.121562927	127.0.0.1	127.0.0.1	TCP	67	10000 → 44210 [PSH, ACK]
13	8.121584320	127.0.0.1	127.0.0.1	TCP	66	44210 → 10000 [ACK] Seq=
14	8.125428035	127.0.0.1	127.0.0.1	TCP	70	10000 → 44210 [PSH, ACK]
15	8.125443727	127.0.0.1	127.0.0.1	TCP	66	44210 → 10000 [ACK] Seq=
16	8.125520855	127.0.0.1	127.0.0.1	TCP	206	10000 → 44210 [PSH, ACK]
17	8.125526308	127.0.0.1	127.0.0.1	TCP	66	44210 → 10000 [ACK] Seq=
18	10.939884974	127.0.0.1	127.0.0.1	UDP	72	51983 → 6868 Len=30
19	10.940666764	127.0.0.1	127.0.0.1	UDP	61	6868 → 51983 Len=19

En la imagen anterior y teniendo en cuenta que el directorio escucha en el puerto 6868 se puede observar perfectamente lo que comentábamos en el párrafo introductorio. Los primeros dos mensajes son un ping y un ping_ok, los dos siguientes un serve y un serve_ok, los dos siguientes un peers interno y un peers ok interno, luego viene una serie de mensajes TCP (del comando peerfiles), por último, observamos un quit y un quit_ok. Procedamos a analizar el contenido de los mensajes para demostrar nuestras conjeturas.

Para no llenar la memoria de capturas de pantalla ilegibles adjunto el enlace de YouTube a una grabación de pantalla mostrando rápidamente el contenido de los mensajes, al igual que haremos en el apartado siguiente:

<https://youtu.be/IYPzdmPB6Fc>

5. Vídeo de ejemplo

Como en el apartado anterior, adjunto enlace a YouTube de la grabación de pantalla mostrando el funcionamiento del programa:

https://youtu.be/snQ7RBsz_V4?si=QDIYHoCIUMTUDxJQ

6. Conclusiones

Ha sido con diferencia el proyecto más difícil en el que hemos trabajado en lo que llevamos de carrera. Nos ha servido para adaptarnos a trabajar como profesionales: empezar con una base implementada por otras personas, que hemos tenido que comprender antes de

empezar a programar; implementar los comandos, en los últimos boletines, con muy poquita guía y mucha libertad para tomar decisiones y también a usar herramientas como GitHub para facilitar el desarrollo del proyecto. En relación con las Redes de Comunicaciones la mayor ventaja que vemos es lo clara que queda la diferencia entre TCP y UDP siendo evidente que el primero es orientado a conexión y que el segundo es multipunto, por ejemplo. La mejora de dirdl es una gran manera de implementar un Stop and Wait funcional. La idea de establecer una probabilidad de corrupción viene muy bien para probar el programa (sobre todo esta última mejora que mencionábamos).

Por otro lado, y a pesar de lo mucho que hemos aprendido, consideramos que el proyecto tiene una carga de trabajo brutal, la asignatura para nada está adecuada para valer solo 6 créditos, ambos coincidimos en que a nivel trabajo es como “una asignatura y media”. Se podría recortar de muchos apartados, pero el más claro es el boletín de los autómatas, se le dedica demasiado tiempo a un apartado puramente de diseño, que luego en la práctica no se respeta y hay que volver a ajustar para que todo cuadre.

En conclusión, un gran proyecto en el que se desarrollan habilidades en las redes de comunicaciones, en programación en Java y en ámbito laboral pero que quita mucho tiempo valioso al alumno.